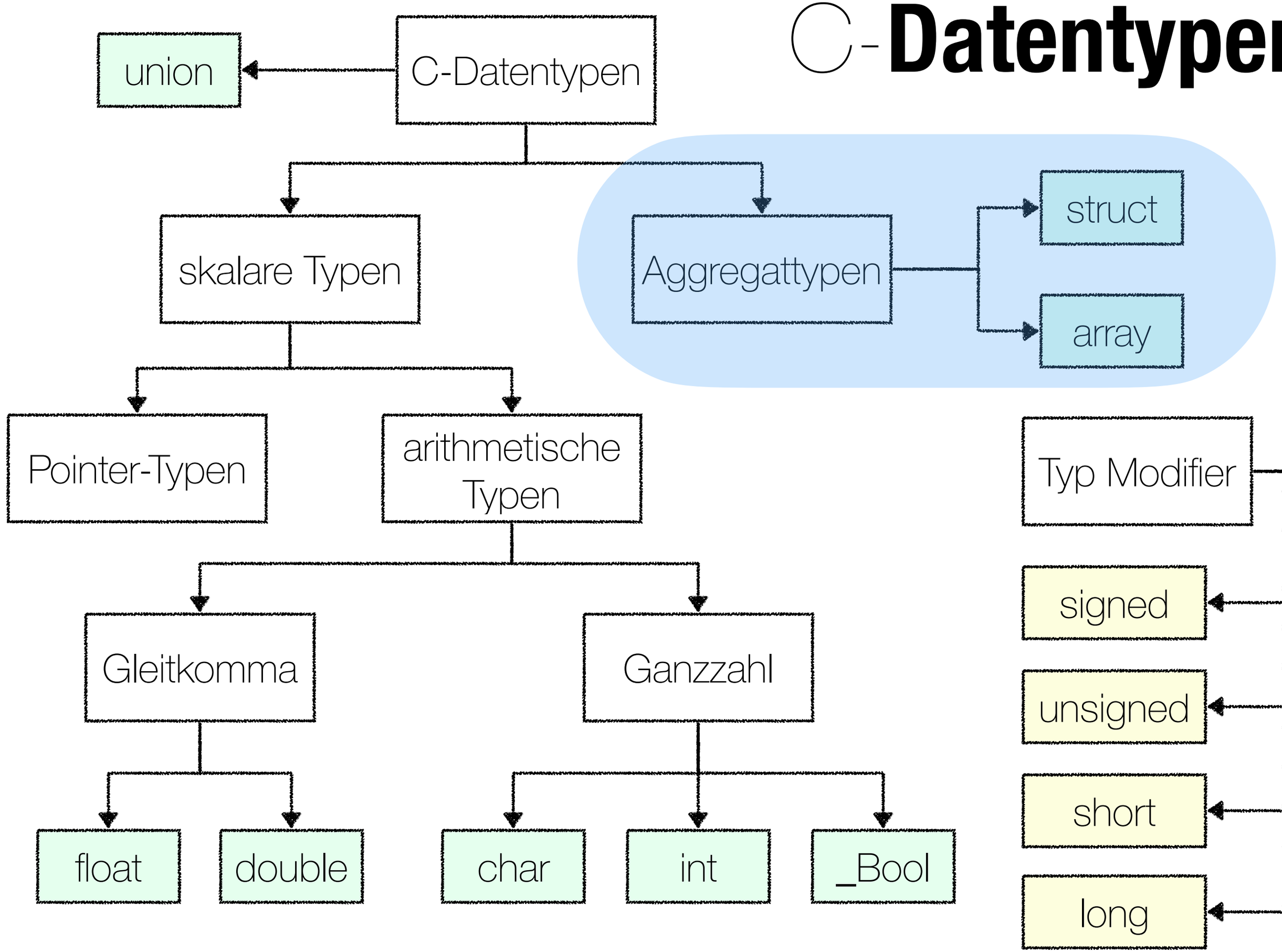
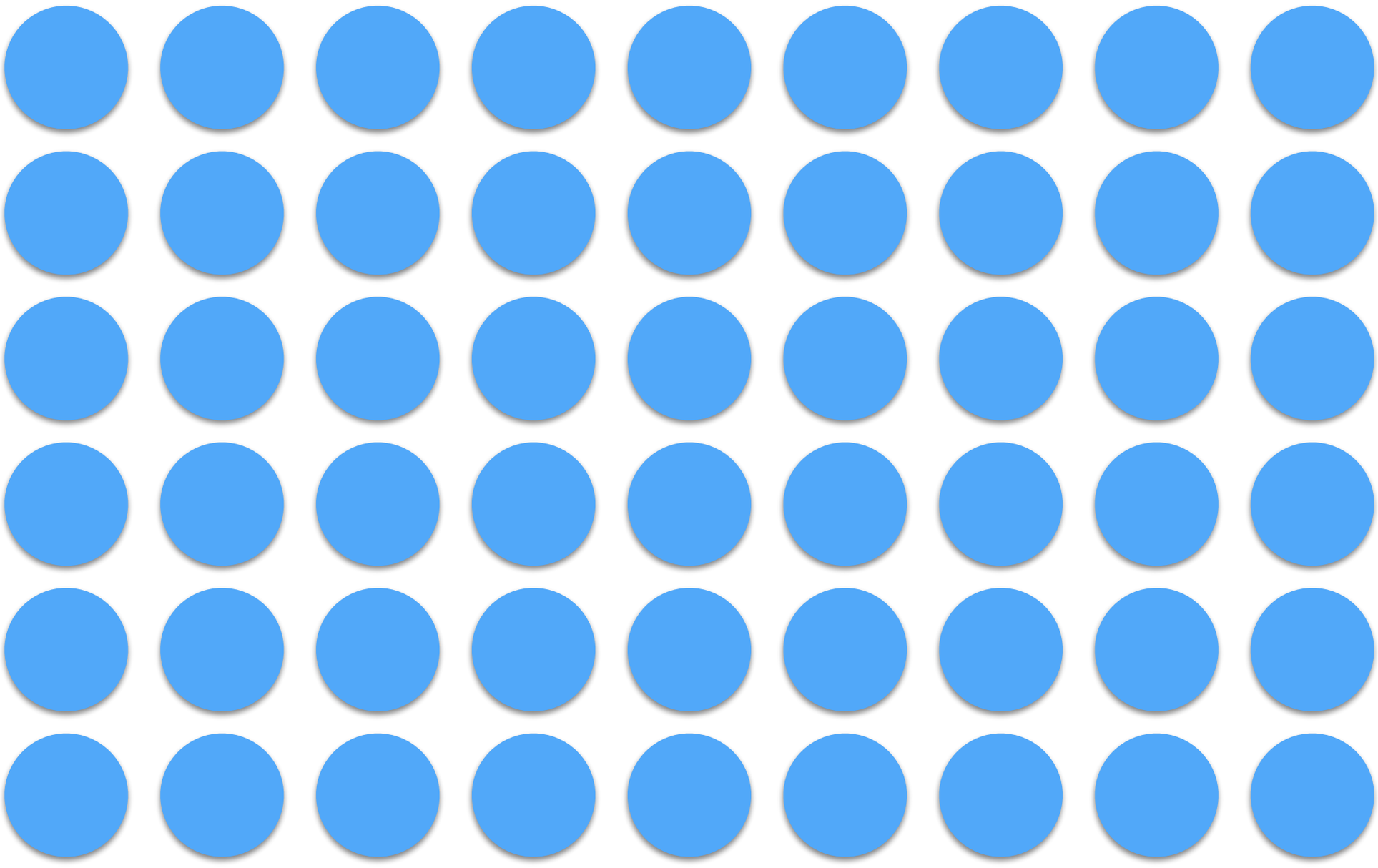
Programmierung 2

Kapitel 10: Aggregattypen

Prof. Dr. Markus Esch
htw saar

C-Datentypen





Arrays

Arrays: Basics

Größe muss angegeben werden

Array-Deklaration

Array-Deklaration und Initialisierung; wird mit Nullen aufgefüllt

```
int main()
{
  float farr[5];
  int iarr [5] = { 1, 3, 5 };
  long larr[] = { 1, 3, 5 };

  for (int i=0 ; i<5 ; ++i) {
    farr[i] = i * 42.42;
    printf("farr[%d] = %f\n", i, farr[i]);
  }
  return 0;
}
```

Implizite Angabe der Feldgröße

Zugriff auf ein Feldelement

```
farr[0] = 0.000000
farr[1] = 42.419998
farr[2] = 84.839996
farr[3] = 127.260002
farr[4] = 169.679993
```

Arrays: Example

- **countDigits.c**

Arrays: Example

```c
int main()
{
  int c, i, nwhite, nother;
  int ndigit[10];
  nwhite = nother = 0;
  for (i = 0; i < 10; ++i)
    ndigit[i] = 0;

  while ((c = getchar()) != '27')
    if (c >= '0' && c <= '9')
      ++ndigit[c-'0'];
    else if (c == ' ' || c == '\n' || c == '\t')
      ++nwhite;
    else
      ++nother;
  }

  printf("digits =");
  for (i = 0; i < 10; ++i)
    printf(" %d", ndigit[i]);
  printf(", white space = %d, other = %d\n", nwhite, nother);
  return 0;
}
```

Initialisierung

ESC

mappen von c auf einen Index zwischen 0 und 9

```c
else if (c = ' ' || c == '\n' || c == '\t')
```

Vorsicht vor Tippfehlern

Array Index out of **Bounds**

kein Bounds Checking!!!


```c
#include <stdio.h>

int main(int argc, const char * argv[])
{
  int arr[3] = {1,2,3};

  for (int i=0 ; i<6 ; ++i)
    printf("arr[%d] = %d\n", i, arr[i]);

  for (int i=0 ; i<6 ; ++i) {
    arr[i] = i * 42;
    printf("arr[%d] = %d\n", i, arr[i]);
  }
  return 0;
}
```

Array Index out of **Bounds**

- **outOfBounds1.c**
- **outOfBounds2.c**
- **outOfBounds3.c**
- **outOfBounds4.c**

Array Index out of **Bounds**

```
int main(int argc, const char * argv[])
{
  int a0[3] = {1,2,3};
  int a1[3] = {3,4,5};
  int a2[3] = {7,8,9};

  for (int i=0 ; i<9 ; ++i)
    printf("a1[%d] = %d\n", i, a1[i]);


  for (int i=0 ; i<9 ; ++i) {
    a1[i] = 42;
    printf("a0[%d] = %2d; a1[%d] = %2d;
      a2[%d] = %d\n", i%3, a0[i%3], i,
      a1[i], i%3, a2[i%3]);
  }
  return 0;
}
```

```
a1[0] = 3
a1[1] = 4
a1[2] = 5
a1[3] = 1
a1[4] = 2
a1[5] = 3
a1[6] = -432865078
a1[7] = 48081803
a1[8] = 1570167608
```

undefiniertes Verhalten!!!

```
a0[0] =  1; a1[0] = 42; a2[0] = 7
a0[1] =  2; a1[1] = 42; a2[1] = 8
a0[2] =  3; a1[2] = 42; a2[2] = 9
a0[0] = 42; a1[3] = 42; a2[0] = 7
a0[1] = 42; a1[4] = 42; a2[1] = 8
a0[2] = 42; a1[5] = 42; a2[2] = 9
a0[0] = 42; a1[6] = 42; a2[0] = 7
a0[1] = 42; a1[7] = 42; a2[1] = 8
a0[2] = 42; a1[8] = 42; a2[2] = 9
```

Achtung! Das Verhalten anderer Systeme kann abweichen!!!

Array Index out of **Bounds**

```c
int main(int argc, const char * argv[])
{
  int a0[3] = {1,2,3};
  int a1[3] = {3,4,5};
  int a2[3] = {7,8,9};

  for(int i=0; i<3;i++)
    printf("a0[%d] addr: %p\n",i, &a0[i]);
  for(int i=0; i<3;i++)
    printf("a1[%d] addr: %p\n",i, &a1[i]);
  for(int i=0; i<3;i++)
    printf("a2[%d] addr: %p\n",i, &a2[i]);

  for (int i=0 ; i<9 ; ++i) {
    a1[i] = 42;
    printf("a0[%d] = %2d; a1[%d] = %2d;
      a2[%d] = %d\n", i%3, a0[i%3], i, a1[i],
      i%3, a2[i%3]);
  }
  return 0;
}
```

Adressoperator

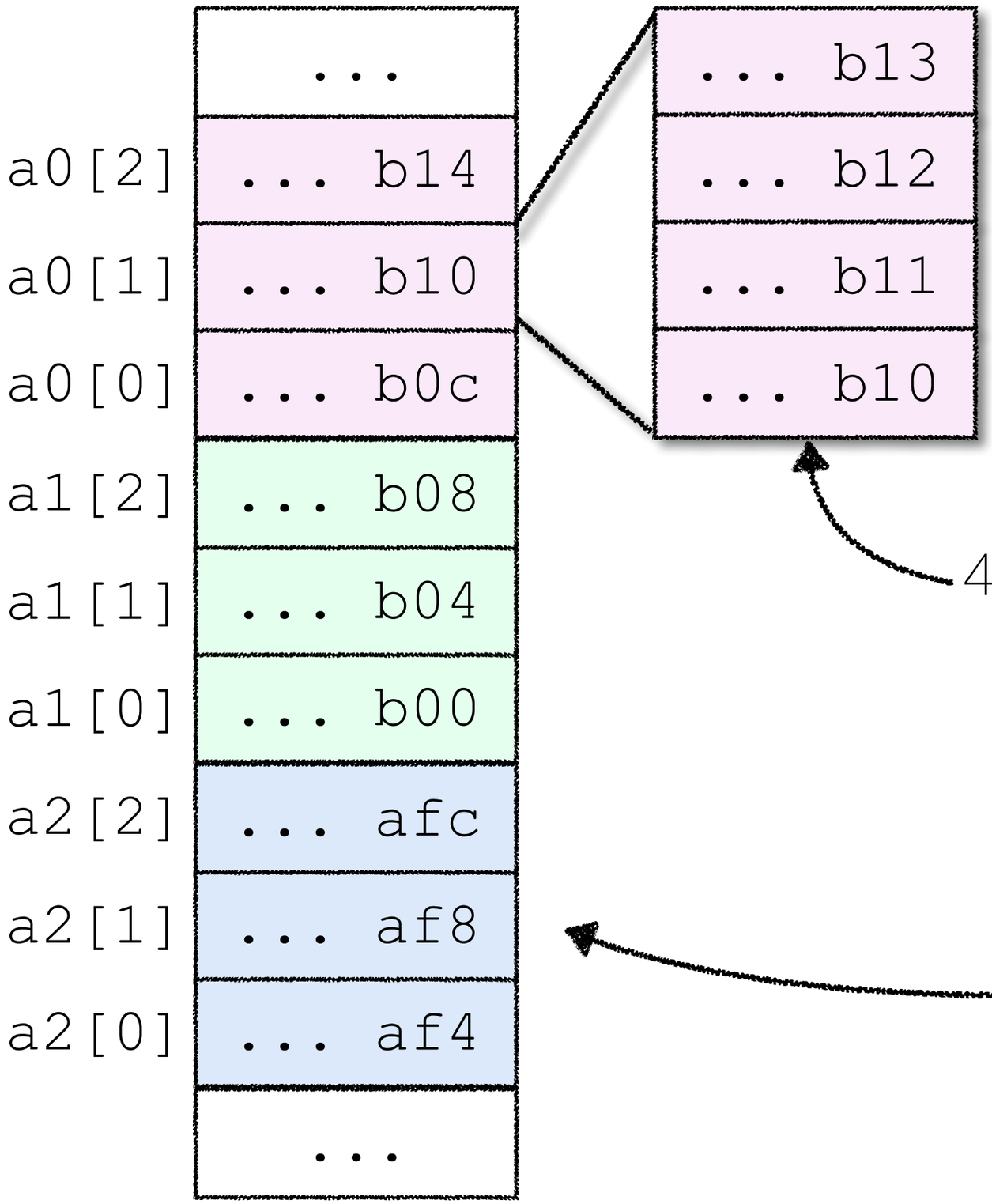
Beispiel!!

```
a0[0] addr: 0x7fff5cee5b0c
a0[1] addr: 0x7fff5cee5b10
a0[2] addr: 0x7fff5cee5b14

a1[0] addr: 0x7fff5cee5b00
a1[1] addr: 0x7fff5cee5b04
a1[2] addr: 0x7fff5cee5b08

a2[0] addr: 0x7fff5cee5af4
a2[1] addr: 0x7fff5cee5af8
a2[2] addr: 0x7fff5cee5afc
```

Array Index out of **Bounds**

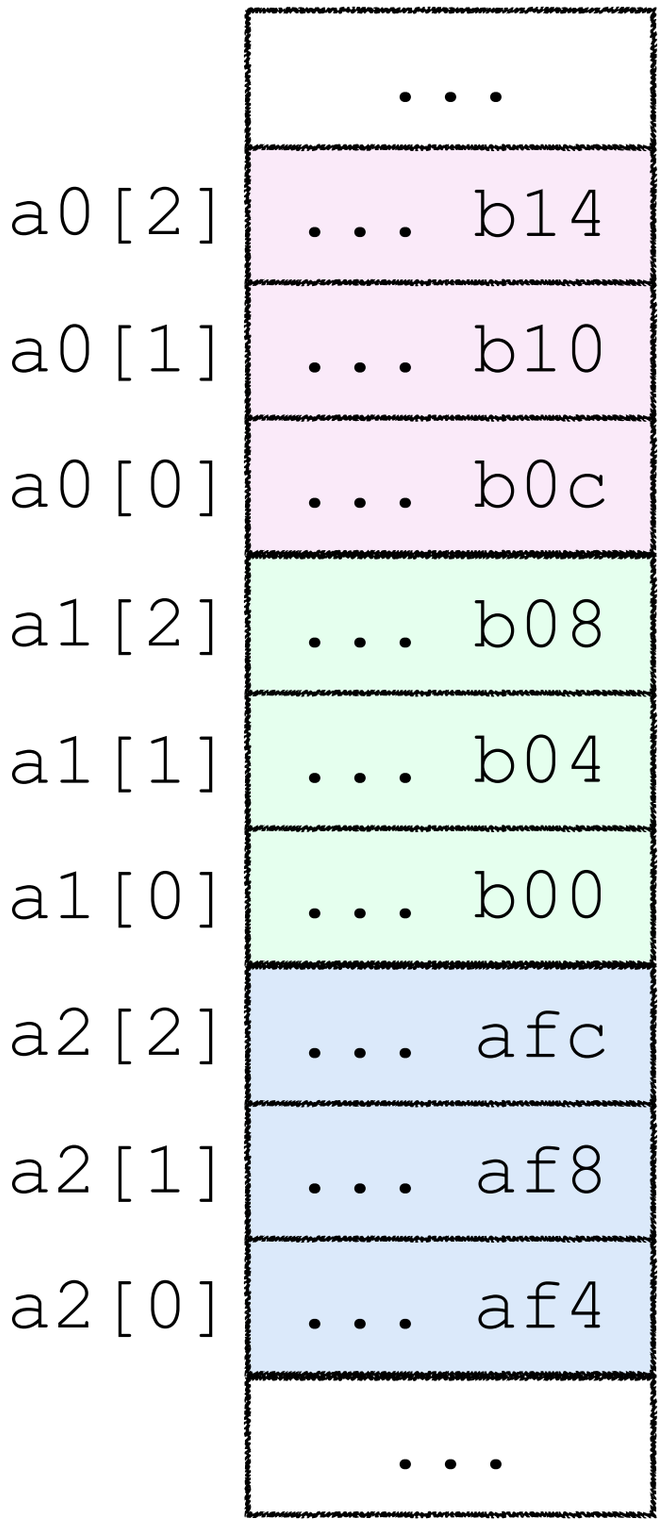



```
a0[0] addr: 0x7fff5cee5b0c
a0[1] addr: 0x7fff5cee5b10
a0[2] addr: 0x7fff5cee5b14

a1[0] addr: 0x7fff5cee5b00
a1[1] addr: 0x7fff5cee5b04
a1[2] addr: 0x7fff5cee5b08

a2[0] addr: 0x7fff5cee5af4
a2[1] addr: 0x7fff5cee5af8
a2[2] addr: 0x7fff5cee5afc
```

Array Index out of **Bounds**

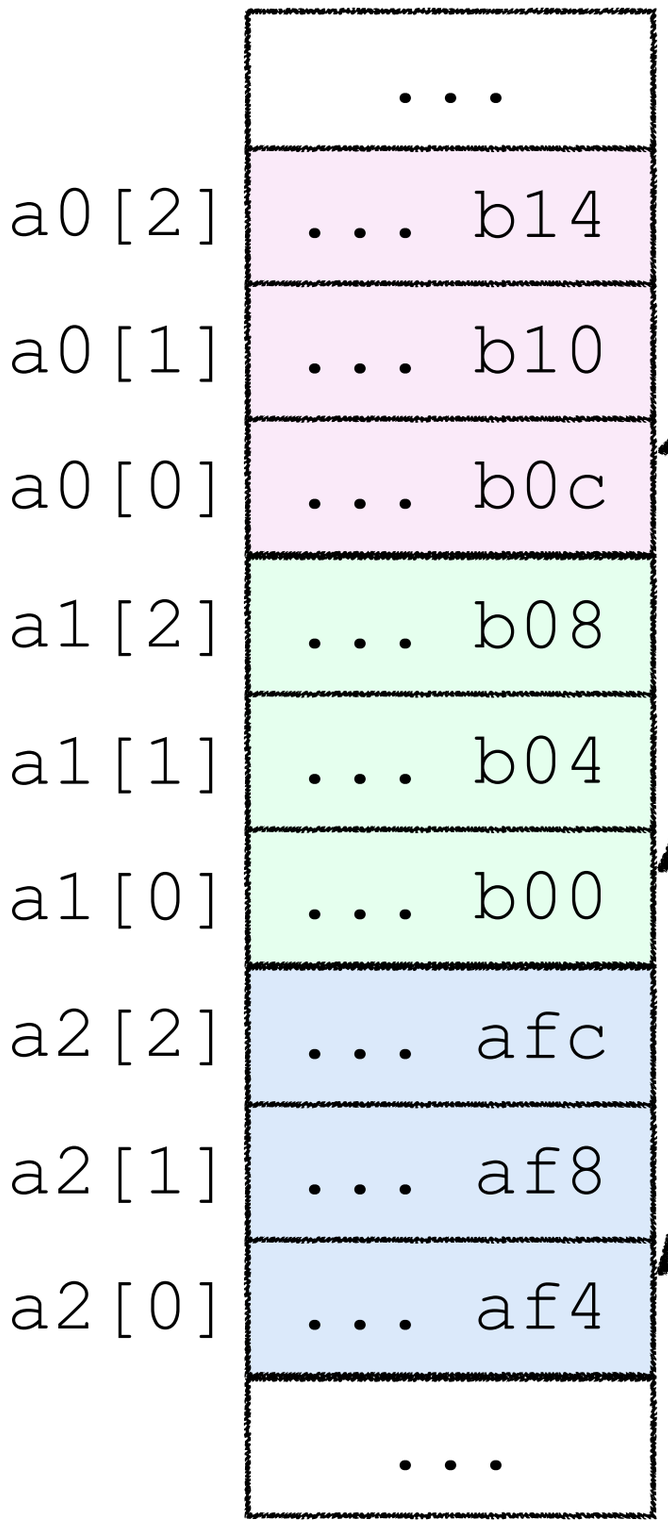



```c
int main(int argc, const char * argv[])
{
  int a0[3] = {1,2,3};
  int a1[3] = {3,4,5};
  int a2[3] = {7,8,9};

  for(int i=0; i<3;i++)
    printf("a0[%d] addr: %p\n",i, &a0[i]);
  for(int i=0; i<3;i++)
    printf("a1[%d] addr: %p\n",i, &a1[i]);
  for(int i=0; i<3;i++)
    printf("a2[%d] addr: %p\n",i, &a2[i]);

  for (int i=0 ; i<9 ; ++i) {
    a1[i] = 42;
    printf("a0[%d] = %2d; a1[%d] = %2d;
      a2[%d] = %d\n", i%3, a0[i%3], i, a1[i],
      i%3, a2[i%3]);
  }
  return 0;
}
```

Array Index out of **Bounds**

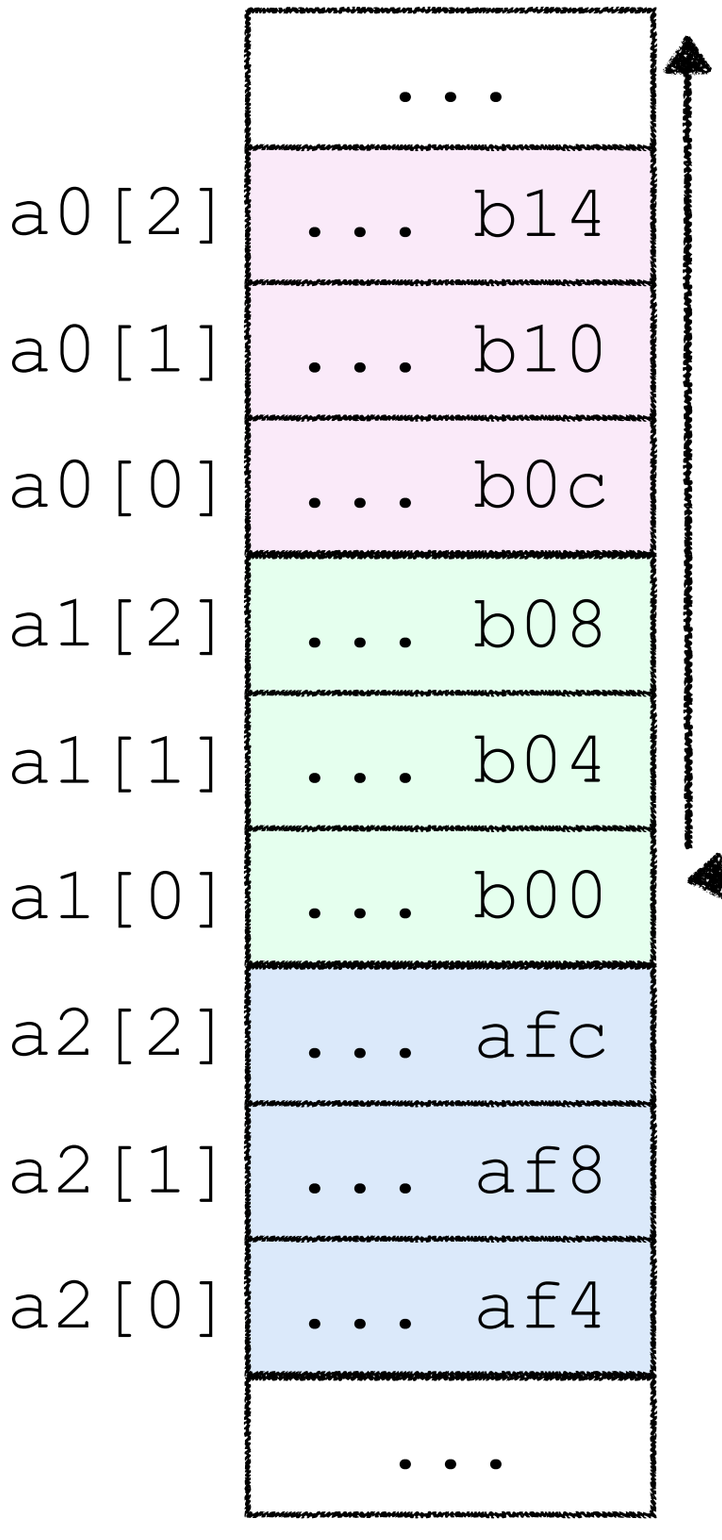



```c
int main(int argc, const char * argv[])
{
  int a0[3] = {1,2,3};
  int a1[3] = {3,4,5};
  int a2[3] = {7,8,9};

  for(int i=0; i<3;i++)
    printf("a0[%d] addr: %p\n",i, &a0[i]);
  for(int i=0; i<3;i++)
    printf("a1[%d] addr: %p\n",i, &a1[i]);
  for(int i=0; i<3;i++)
    printf("a2[%d] addr: %p\n",i, &a2[i]);

  for (int i=0 ; i<9 ; ++i) {
    a1[i] = 42;
    printf("a0[%d] = %2d; a1[%d] = %2d;
      a2[%d] = %d\n", i%3, a0[i%3], i, a1[i],
      i%3, a2[i%3]);
  }
  return 0;
}
```

Array Index out of **Bounds**

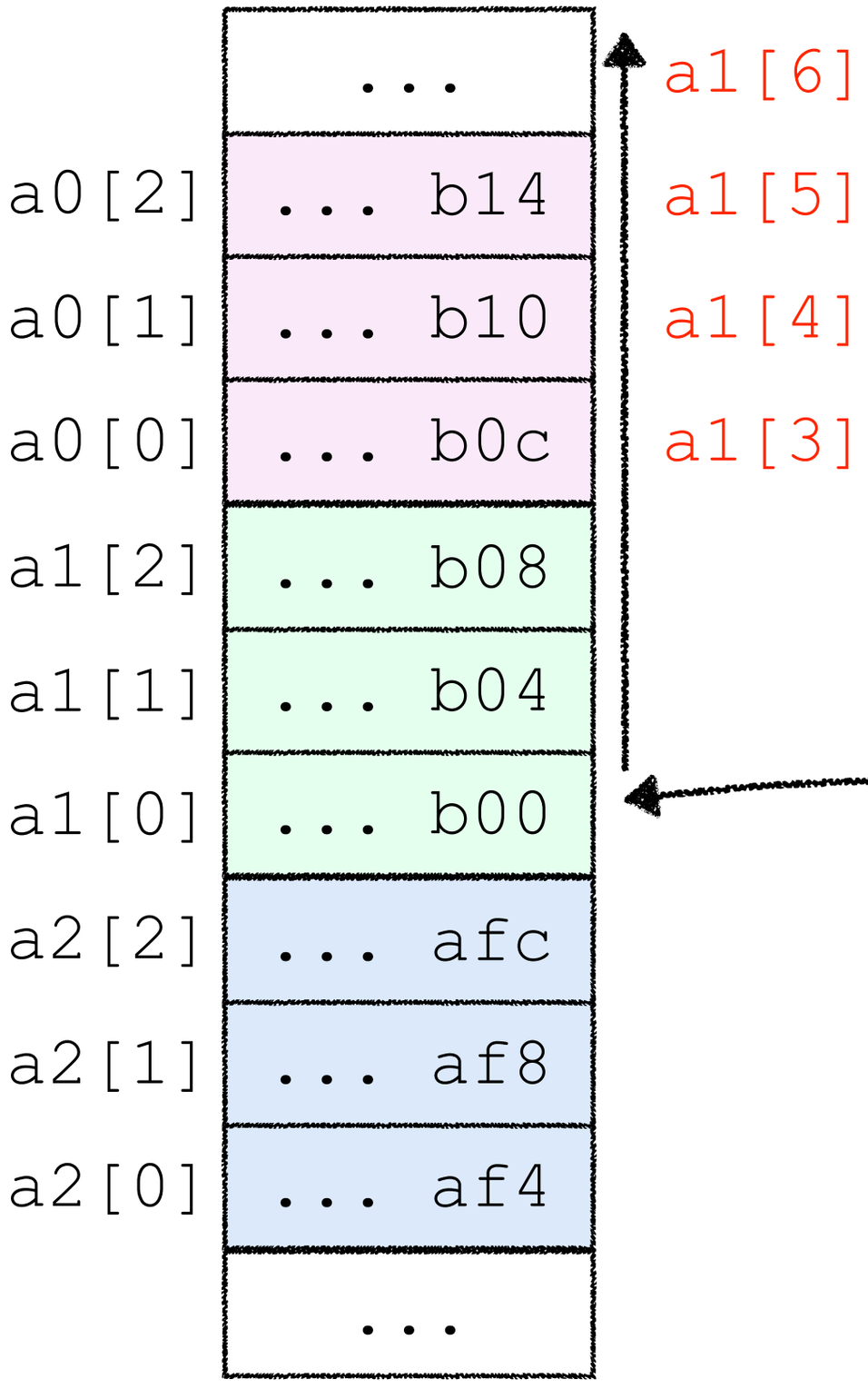



```c
int main(int argc, const char * argv[])
{
  int a0[3] = {1,2,3};
  int a1[3] = {3,4,5};
  int a2[3] = {7,8,9};

  for(int i=0; i<3;i++)
    printf("a0[%d] addr: %p\n",i, &a0[i]);
  for(int i=0; i<3;i++)
    printf("a1[%d] addr: %p\n",i, &a1[i]);
  for(int i=0; i<3;i++)
    printf("a2[%d] addr: %p\n",i, &a2[i]);

  for (int i=0 ; i<9 ; ++i) {
    a1[i] = 42;
    printf("a0[%d] = %2d; a1[%d] = %2d;
      a2[%d] = %d\n", i%3, a0[i%3], i, a1[i],
      i%3, a2[i%3]);
  }
  return 0;
}
```

Array Index out of **Bounds**

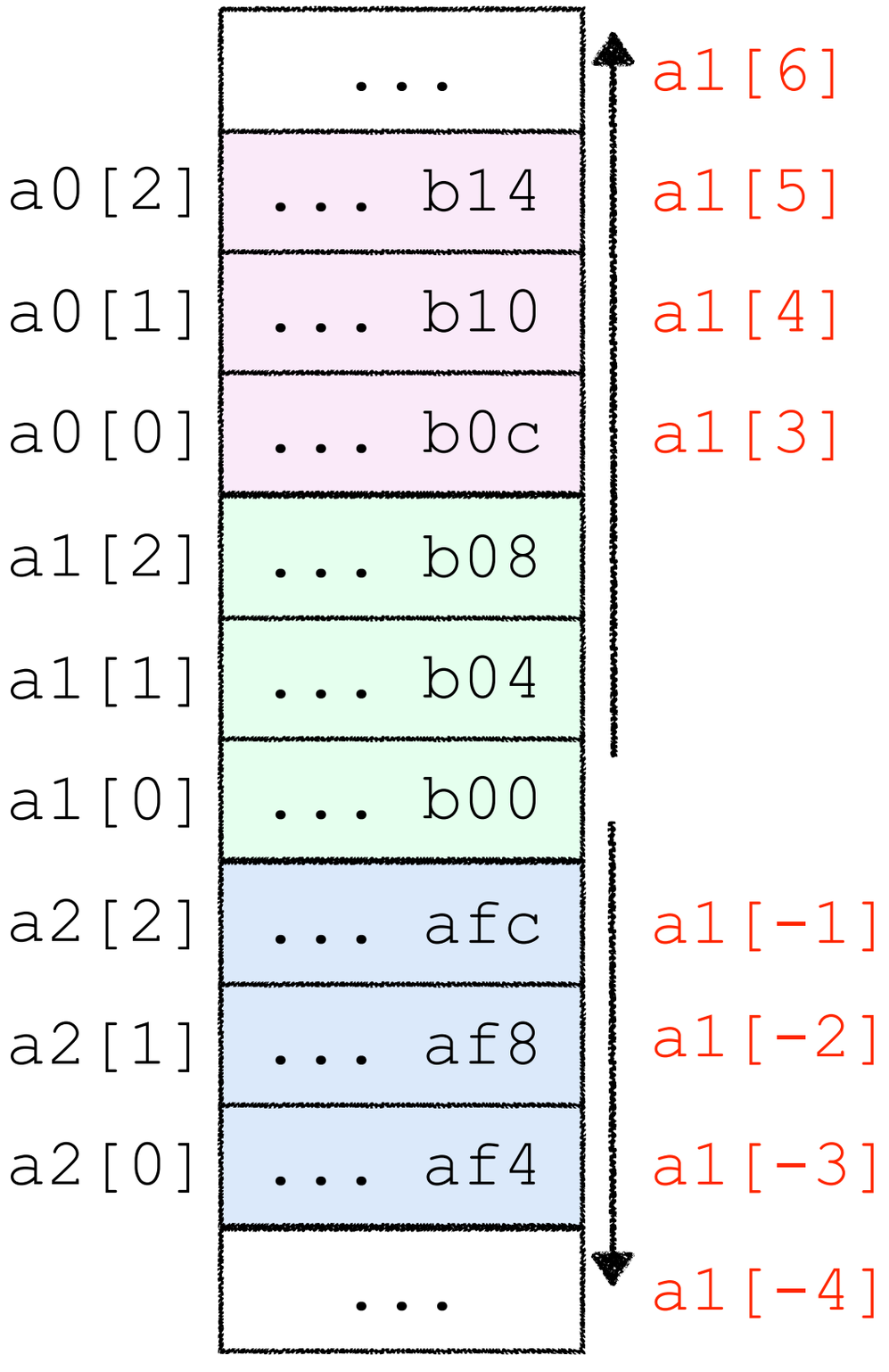



```c
int main(int argc, const char * argv[])
{
  int a0[3] = {1,2,3};
  int a1[3] = {3,4,5};
  int a2[3] = {7,8,9};

  for(int i=0; i<3;i++)
    printf("a0[%d] addr: %p\n",i, &a0[i]);
  for(int i=0; i<3;i++)
    printf("a1[%d] addr: %p\n",i, &a1[i]);
  for(int i=0; i<3;i++)
    printf("a2[%d] addr: %p\n",i, &a2[i]);

  for (int i=0 ; i<9 ; ++i) {
    a1[i] = 42;
    printf("a0[%d] = %2d; a1[%d] = %2d;
      a2[%d] = %d\n", i%3, a0[i%3], i, a1[i],
      i%3, a2[i%3]);
  }
  return 0;
}
```

Array Index out of **Bounds**



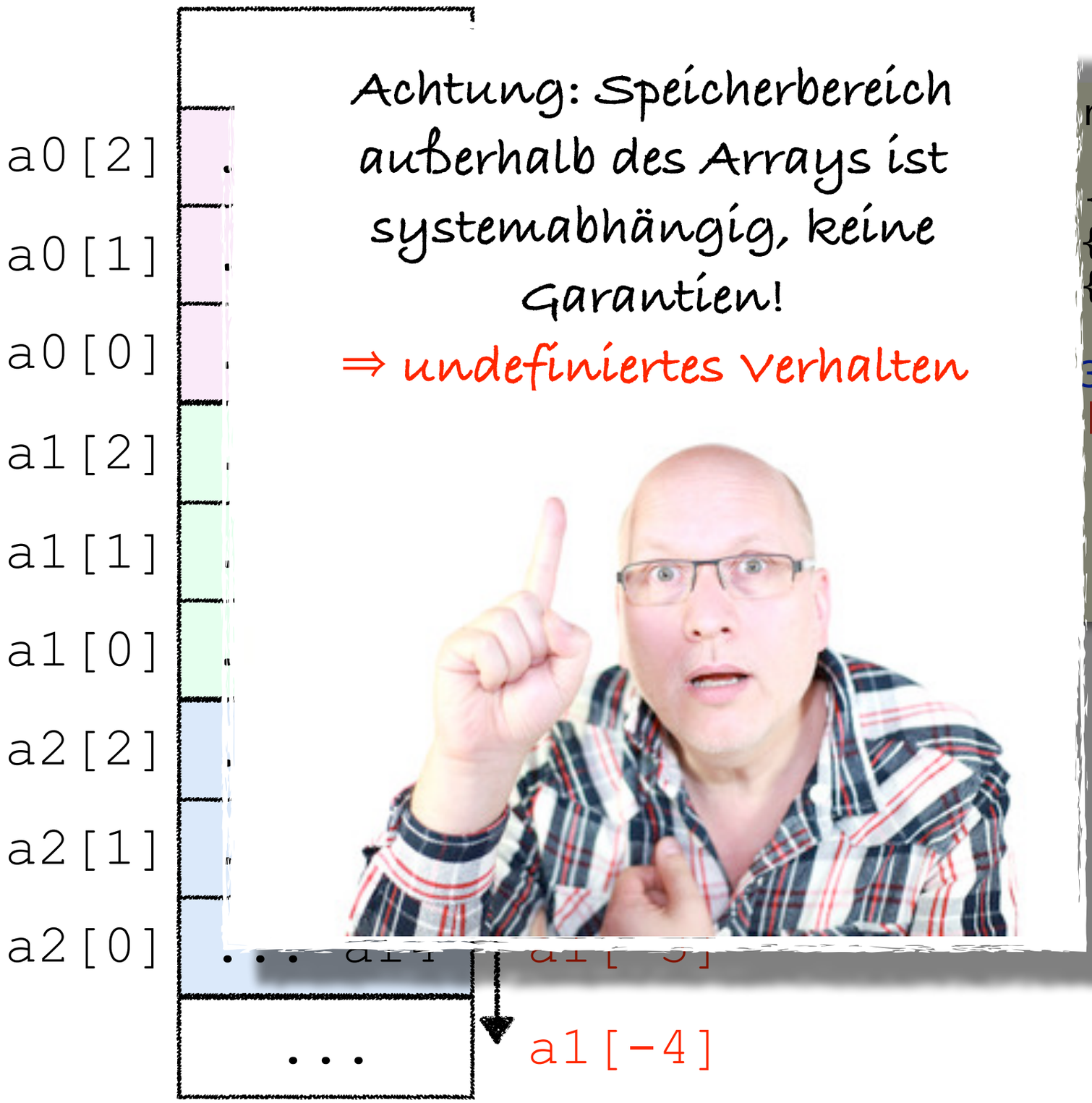

```c
int main(int argc, const char * argv[])
{
   int a0[3] = {1,2,3};
   int a1[3] = {4,5,6};
   int a2[3] = {7,8,9};

   for (int i=-3 ; i<7 ; i++) {
     printf("a1[%d] = %d\n", i, a1[i]);
   }

   return 0;
}
```

```
a1[-3] = 7
a1[-2] = 8
a1[-1] = 9
a1[0] = 4
a1[1] = 5
a1[2] = 6
a1[3] = 1
a1[4] = 2
a1[5] = 3
```

Array Index out of **Bounds**



Achtung: Speicherbereich außerhalb des Arrays ist systemabhängig, keine Garantien!

⇒ undefiniertes Verhalten


```
rgc, const char * argv[])

{1,2,3};
{4,5,6};
{7,8,9};

3 ; i<7 ; i++) {
[%d] = %d\n", i, a1[i]);
```

```
a1[-3] = 7
a1[-2] = 8
a1[-1] = 9
a1[0] = 4
a1[1] = 5
a1[2] = 6
a1[3] = 1
a1[4] = 2
a1[5] = 3
```

Anzahl Elemente in einem Array

JAVA: `a.length`

?

```c
#include <stdio.h>

int main()
{
   int a[] = { 20, 21, 22, 24, 25 };

   for(int i =0; i<ANZAHL_ELMENTE; i++)
     printf("a[%d] = %d\n", i, a[i]);
   return 0;
}
```

Anzahl Elemente in einem Array

20 byte

4 byte

```c
#include <stdio.h>

int main()
{
   int a[] = { 20, 21, 22, 24, 25 };

   for(int i =0; i < sizeof(a)/sizeof(a[0]); i++)
      printf("a[%d] = %d\n", i, a[i]);

   return 0;
}
```

Anzahl Elemente in einem Array

funktioniert hier **nicht**!!
…später mehr

```c
#include <stdio.h>

void printArray(int a[]){
  for(int i =0; i < sizeof(a)/sizeof(a[0]); i++)
    printf("a[%d] = %d\n", i, a[i]);
}

int main()
{
  int a[] = { 20, 21, 22, 24, 25 };
  for(int i =0; i < sizeof(a)/sizeof(a[0]); i++)
    printf("a[%d] = %d\n", i, a[i]);

  printArray(a);
  return 0;
}
```

Mehrdimensionale Arrays

Zugriff auf ein Array-Element

Achtung!
nicht so:

```
tda[row, col]
```

```c
#include <stdio.h>

#define X 2
#define Y 3

void printTwoDimArr(int tda[][Y])
{
  int row, col;
  for (row = 0 ; row < X ; ++row)
    for (col = 0 ; col < Y ; ++col)
      printf("twoDimArr[%d][%d] = %d\n", row, col, tda[row][col]);
}

int main()
{
  int twoDimArr[2][3] = { {10, 11, 12}, {20, 21, 22} };
  printTwoDimArr(twoDimArr);
  return 0;
}
```

```
twoDimArr[0][0] = 10
twoDimArr[0][1] = 11
twoDimArr[0][2] = 12
twoDimArr[1][0] = 20
twoDimArr[1][1] = 21
twoDimArr[1][2] = 22
```

Deklaration und Initialisierung

Mehrdimensionale Arrays

```c
#include <stdio.h>
#define X 2
#define Y 3

void printTwoDimArrFormatted(int tda[][Y])
{
  int row, col;
  printf("      ");
  for (col = 0 ; col < Y ; ++col)
    printf("[%d]  ",col);
  printf("\n");
  for (row = 0 ; row < X ; ++row){
    printf("[%d]  ",row);
    for (col = 0 ; col < Y ; ++col)
      printf(" %d  ", tda[row][col]);
    printf("\n");
  }
}

int main()
{
  int twoDimArr[2][3] = { {10, 11, 12}, {20, 21, 22} };
  printTwoDimArr(twoDimArr);
  return 0;
}
```

Größe der zweiten Dimension muss spezifiziert werden!

```
      [0]  [1]  [2]
[0]    10   11   12
[1]    20   21   22
```

Speicherlayout

- **TwoDimArrayMemLayout.c**
- **MultiDimArray.c**

Speicherlayout

```c
#include <stdio.h>
#define X 2
#define Y 3

void printAddresses(int a[][Y], int row, int col){
  printf("      ");
  for (col = 0 ; col < Y ; ++col)
    printf("[%d]                ",col);
  printf("\n");
  for (row = 0 ; row < X ; ++row){
    printf("[%d]  “,row);
    for (col = 0 ; col < Y ; ++col)
      printf(" %p  ", &a[row][col]);
    printf("\n");
  }
}

int main()
{
   int a[X][Y] = { {10, 11, 12}, {20, 21, 22} };
    printAddresses(a, X, Y);
    return 0;
}
```

Beispiel

```
      [0]                 [1]                 [2]
[0]   0x7fff50272b20   0x7fff50272b24   0x7fff50272b28
[1]   0x7fff50272b2c   0x7fff50272b30   0x7fff50272b34
```

Speicherlayout

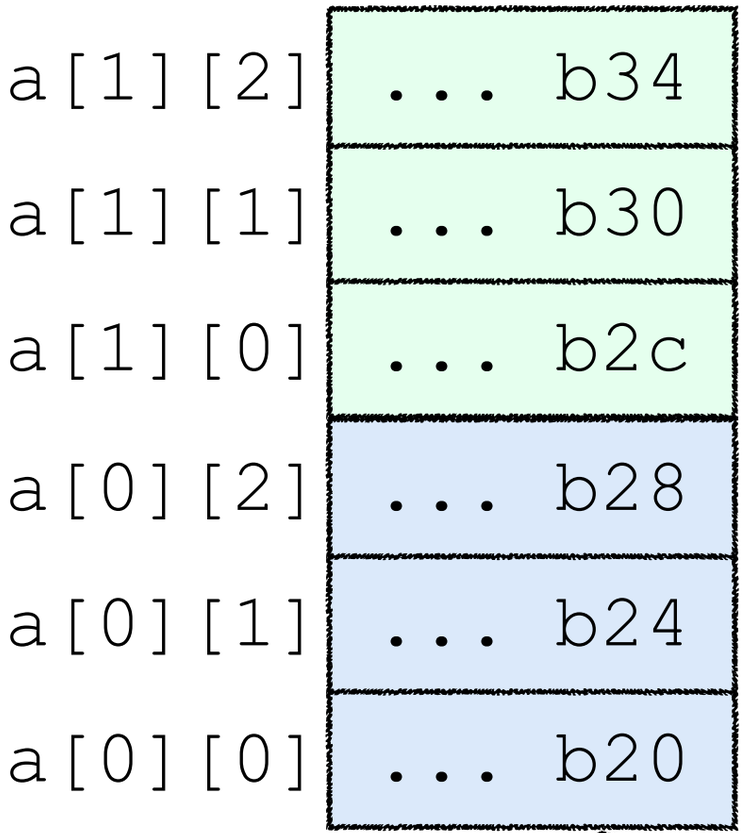
```c
#include <stdio.h>
#define X 2
#define Y 3

void printAddresses(int a[][Y], int row, int col){
  printf("      ");
  for (col = 0 ; col < Y ; ++col)
    printf("[%d]                 ",col);
  printf("\n");
  for (row = 0 ; row < X ; ++row){
    printf("[%d]  “,row);
    for (col = 0 ; col < Y ; ++col)
      printf(" %p  ", &a[row][col]);
    printf("\n");
  }
}

int main()
{
   int a[X][Y] = { {10, 11, 12}, {20, 21, 22} };
    printAddresses(a, X, Y);
    return 0;
}
```

Beispiel

```
      [0]                 [1]                 [2]
[0]   0x7fff50272b20   0x7fff50272b24   0x7fff50272b28
[1]   0x7fff50272b2c   0x7fff50272b30   0x7fff50272b34
```

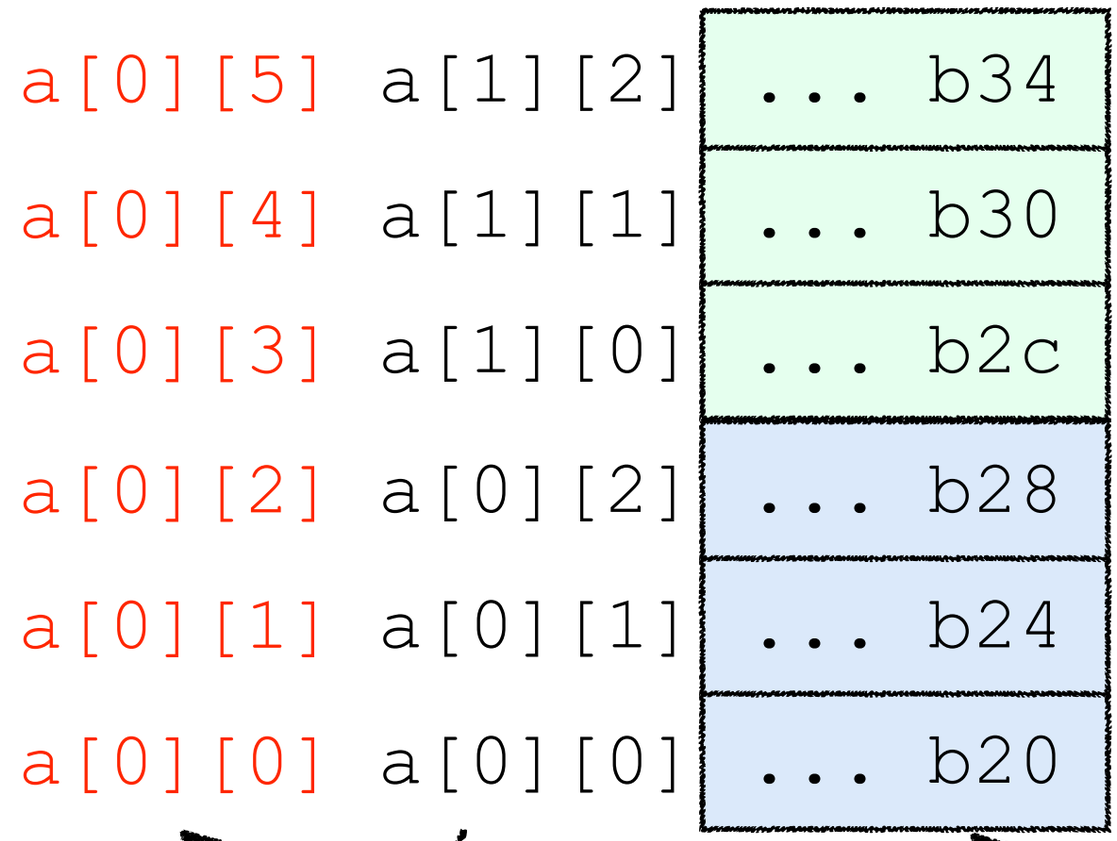


Speicherlayout

Allgemein:

`a[x][0]=a[0][x * columns]`

`a[x][y]=a[0][x * columns + y]`



	[0]	[1]	[2]
[0]	0x7fff50272b20	0x7fff50272b24	0x7fff50272b28
[1]	0x7fff50272b2c	0x7fff50272b30	0x7fff50272b34

Speicherlayout

Allgemein:

`a[x][0]=a[0][x * columns]`

`a[x][y]=a[0][x * columns + y]`

```
tda[0][0] = 10; tda[0][0 * 3 + 0] = 10
tda[0][1] = 11; tda[0][0 * 3 + 1] = 11
tda[0][2] = 12; tda[0][0 * 3 + 2] = 12
tda[1][0] = 20; tda[0][1 * 3 + 0] = 20
tda[1][1] = 21; tda[0][1 * 3 + 1] = 21
tda[1][2] = 22; tda[0][1 * 3 + 2] = 22
```

```
void printTwoDimArr(int tda[][Y], int rows, int cols){
  for (int row = 0 ; row < rows ; ++row){
    for (int col = 0 ; col < cols ; ++col){
      printf("tda[%d][%d] = %d; tda[0][%d * %d + %d] =  %d\n",
        row, col, tda[row][col], row, cols,
        col, tda[0][row * cols + col]);
    }
  }
}
```

Speicherlayout

```c
#include <stdio.h>

#define X 2
#define Y 3

void printAsOneDimArr(int oneDimArr[], int size)
{
  for (int i = 0 ; i < size ; ++i){
    printf("oneDimArr[%d] = %d\n", i, oneDimArr[i]);
  }
}

int main()
{
  int twoDimArr[X][Y] = { {10, 11, 12}, {20, 21, 22} };
  printAsOneDimArr(twoDimArr[0], X * Y);
  printTwoDimArr(twoDimArr);
  return 0;
}
```

```
oneDimArr[0] = 10
oneDimArr[1] = 11
oneDimArr[2] = 12
oneDimArr[3] = 20
oneDimArr[4] = 21
oneDimArr[5] = 22
```

6

```
      [0]   [1]   [2]
[0]    10    11    12
[1]    20    21    22
```

Mehr **Dimensionen**

drei Dimensionen

```c
#include <stdio.h>
#define X 2
#define Y 3
#define Z 2

int main()
{
   int a[X][Y][Z] =
     {
       { {10, 11}, {20, 21}, {30, 31} },
       { {40, 41}, {50, 51}, {60, 61} }
     };
   int m = 0;
   for(int i = 0; i<X; i++)
     for(int j=0; j<Y; j++)
       for(int k=0; k<Z; k++){
         printf("%d\n",i, j, k, a[i][j][k]);
         printf("%d\n",i, j, k, a[0][0][m++]);
         printf("%p\n\n",i, j, k, &a[i][j][k]);
       }
   return 0;
}
```

char-Arrays

Strings sind char-Arrays

Zuweisung eines konstanten Strings

```c
#include <stdio.h>

int main()
{
  char str[] = "simple string";
  printf("str: '%s' has length: %lu\n", str, sizeof(str));
  for (int i=0 ; i<sizeof(str) ; ++i)
    printf("str[%d]: %c  ", i, str[i]);
  printf("\n");
  return 0;
}
```

?

```
str: 'simple string' has length: 14
str[0]: s  str[1]: i  str[2]: m
str[3]: p  str[4]: l  str[5]: e
str[6]:    str[7]: s  str[8]: t
str[9]: r  str[10]: i  str[11]: n
str[12]: g  str[13]:
```

Array Index out of **Bounds**

- **SimpleString.c**
- **StringFailure.c**

char-Arrays

- jeder String endet mit **`'\0'` !!!**

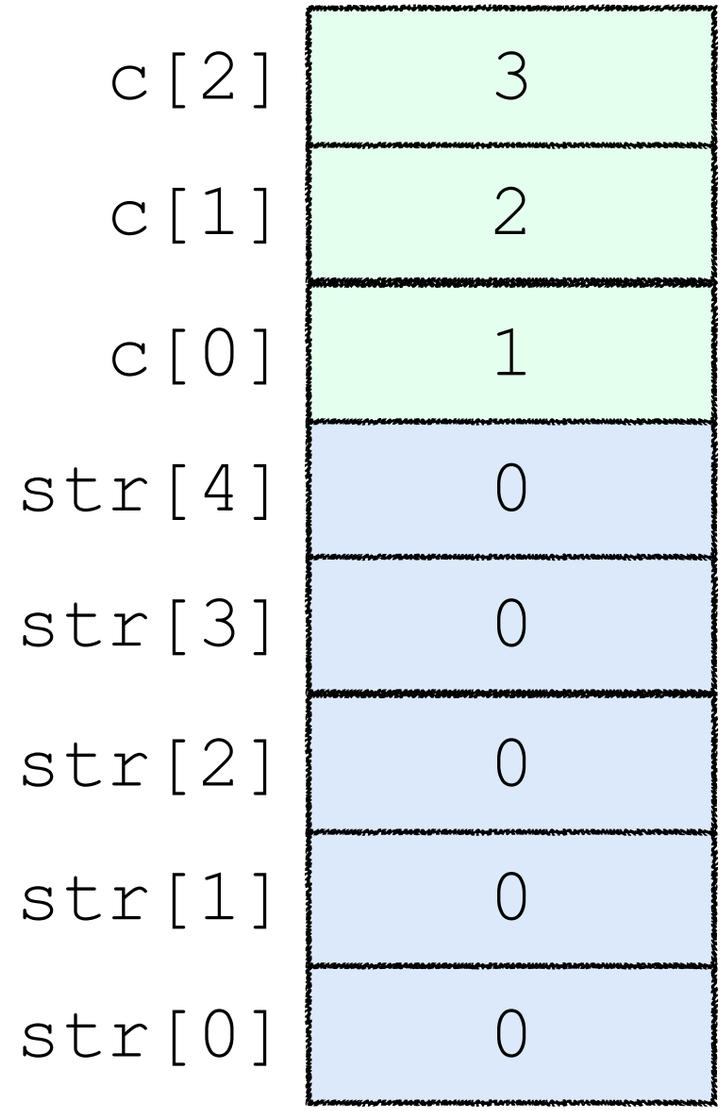
```
void stringcopy(char c1[], char c2[], int c1Size){
  for(int i=0; i<c1Size; i++){
    c2[i]=c1[i];
    printf("c1[%d]: %c\n", i, c1[i]);
  }
}

int main(int argc, const char * argv[]){
  char c[3] = {1,2,3};
  char str[5] = {0};
  char hello[] = "Hello";

  stringcopy(hello, str, sizeof(hello));

  for (int i=0 ; i<3 ; ++i)
    printf("i[%d]: %d\n", i, c[i]);
  return 0;
}
```

```
i[0]: 0
i[1]: 2
i[2]: 3
```



char-Arrays

- jeder String endet mit **`'\0'` !!!**

```
void stringcopy(char c1[], char c2[], int c1Size){
  for(int i=0; i<c1Size; i++){
    c2[i]=c1[i];
    printf("c1[%d]: %c\n", i, c1[i]);
  }
}

int main(int argc, const char * argv[]){
  char c[3] = {1,2,3};
  char str[5] = {0};
  char hello[] = "Hello";

  stringcopy(hello, str, sizeof(hello));

  for (int i=0 ; i<3 ; ++i)
    printf("i[%d]: %d\n", i, c[i]);
  return 0;
}
```

Hello\0

```
i[0]: 0
i[1]: 2
i[2]: 3
```

c[2]	3
c[1]	2
c[0]	\0
str[4]	o
str[3]	l
str[2]	l
str[1]	e
str[0]	H

Structs

Structs: Basics

- ein **Struct** fasst mehrere Variablen zu einer logischen Einheit zusammen
  - die Variablen können unterschiedliche **Datentypen** haben

- Variablen eine Structs bezeichnen wir als **Komponenten**

Structs: Basics

Keyword

Bezeichner

```
struct date
{
  char day;
  char month;
  short year;
};

int main()
{
  struct date today;
}
```

Komponenten

Deklaration eines Structs

erzeugen einer Variablen

Structs: Basics

Keyword

Bezeichner

```
struct date
{
  char day;
  char month;
  short year;
}birthday, easter;

int main()
{
  struct date today;
}
```

Deklaration eines Structs

Komponenten

erzeugen einer Variablen

erzeugen einer Variablen

Structs: Basics

```c
#include<stdio.h>
struct date
{
  char day;
  char month;
  short year;
}today, piday = {14, 03, 1988};

int main()
{
  struct date towelday = {25, 5, 2001};

  printf("today: %d.%d.%d\n",
     towelday.day,
     towelday.month,
     towelday.year);

  today = piday;
}
```

Initialisierung

Zuweisung

Zugriff auf Komponenten

Structs und **Typedef**

```c
struct date{
  char day;
  char month;
  short year;
};

int main()
{
  struct date piday = {14, 3, 1988};
}
```

```c
typedef struct {
  char day;
  char month;
  short year;
} Date;

int main()
{
  Date piday = {14, 3, 1988};
}
```

Array aus Structs

```c
#include<stdio.h>

typedef struct {
  char name[50];
  int age;
}Person;


int main(){
  Person person[10];
  int i;
  for(i=0; i<10; i++) {
    person[i].age = 20 + i;
    printf("%d ", person[i].age);
  }
}
```

erzeugen eines Arrays aus Structs

Zugriff auf die Elemente

Call by ... ?

- **callBy1.c**
- **callBy2.c**

Call by . . . ?

```c
typedef struct { int i; }someStruct;

void structFunction(someStruct s){ s.i = 42; }

void arrayFunction(int a[]) { a[0] = 42; }

void intFunction(int i){   i = 42; }

int main()
{
   int i = 23;
   someStruct s = { 23 };
   int a[1] = { 23 };
   printf("i:%d; s.i: %d; a[0]: %d\n”, i, s.i, a[0]);
   intFunction(i);
   structFunction(s);
   arrayFunction(a);
   printf("i:%d; s.i: %d; a[0]: %d\n", i, s.i, a[0]);
   return 0;
}
```

Call by . . . ?

```c
typedef struct { int i; }someStruct;

void structFunction(someStruct s){ s.i = 42; }

void arrayFunction(int a[]) { a[0] = 42; }

void intFunction(int i){   i = 42; }

int main()
{
   int i = 23;
   someStruct s = { 23 };
   int a[1] = { 23 };
   printf("i:%d; s.i: %d; a[0]: %d\n”, i, s.i, a[0]);
   intFunction(i);
   structFunction(s);
   arrayFunction(a);
   printf("i:%d; s.i: %d; a[0]: %d\n", i, s.i, a[0]);
   return 0;
}
```

```
i:23; s.i: 23; a[0]: 23
i:23; s.i: 23; a[0]: 42
```

Call by . . . ?

```c
typedef struct { int i; }someStruct;

void structFunction(someStruct s){ s.i = 42; }

void arrayFunction(int a[]) { a[0] = 42; }

void intFunction(int i){   i = 42; }

int main()
{
   int i = 23;
   someStruct s = { 23 };
   int a[1] = { 23 };
   printf("i:%d; s.i: %d; a[0]: %d\n”, i, s.i, a[0]);
   intFunction(i);
   structFunction(s);
   arrayFunction(a);
   printf("i:%d; s.i: %d; a[0]: %d\n", i, s.i, a[0]);
   return 0;
}
```

call by value

```
i:23; s.i: 23; a[0]: 23
i:23; s.i: 23; a[0]: 42
```

Call by . . . ?

```c
typedef struct { int i; }someStruct;

void structFunction(someStruct s){ s.i = 42; }

void arrayFunction(int a[]) { a[0] = 42; }

void intFunction(int i){   i = 42; }

int main()
{
   int i = 23;
   someStruct s = { 23 };
   int a[1] = { 23 };
   printf("i:%d; s.i: %d; a[0]: %d\n”, i, s.i, a[0]);
   intFunction(i);
   structFunction(s);
   arrayFunction(a);
   printf("i:%d; s.i: %d; a[0]: %d\n", i, s.i, a[0]);
   return 0;
}
```

```
i:23; s.i: 23; a[0]: 23
i:23; s.i: 23; a[0]: 42
```

call by reference!

Exkurs: Call **by ...**

- Call by Value
  - Definition: Übergabe von Argumenten als Kopie der Werte.
  - Verhalten: Änderungen an den Parametern innerhalb der Funktion beeinflussen nicht die ursprünglichen Variablen.

- Call by Reference
  - Übergabe von Argumenten als Referenz.
  - Verhalten: Änderungen an den Parametern innerhalb der Funktion beeinflussen die ursprünglichen Variablen.

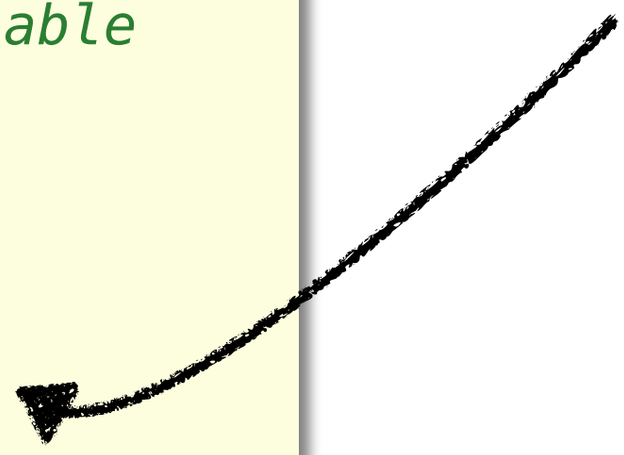
Exkurs: Call **by ...**

```c
void modifyValue(int num) {
  num = 10; // Ändert nur die lokale Kopie
}

int main() {
  int x = 5;
  modifyValue(x);
  // x bleibt 5
}
```

```c
void modifyValue(int *num) {
  *num = 10; // Ändert den Wert der
             // ursprünglichen Variable
}

int main() {
  int x = 5;
  modifyValue(&x);
  // x wird 10
}
```



Trotzdem
call by value, da
der Wert
des Pointers
kopiert wird.

Exkurs: Call **by ...**

```java
public class CallByValueExample {
  static class Example {
    int value;
  }

  public static void modifyObject(Example obj) {
    obj.value = 10; // Ändert das Originalobjekt
  }

  public static void modifyPrimitive(int num) {
    num = 10; // Ändert nur die lokale Kopie
  }

  public static void main(String[] args) {
    Example example = new Example();
    example.value = 5;
    modifyObject(example);
    System.out.println("Nach modifyObject: example.value
                        = " + example.value);
    // Ausgabe: example.value ist jetzt 10

    int x = 5;
    modifyPrimitive(x);
    System.out.println("Nach modifyPrimitive: x = " + x);
    // Ausgabe: x bleibt 5
  }
}
```

Trotzdem call by value, da die Referenz kopiert wird.

Was haben Sie **gelernt**?

Erklären Sie ihrem Sitznachbarn (45 Sekunden):

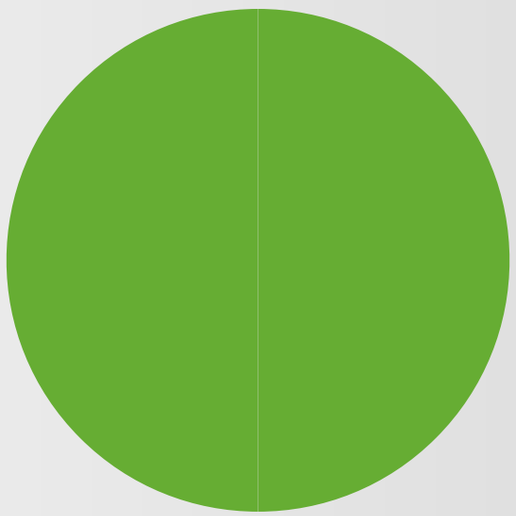
- Wie verwendet man Arrays in C und was ist zu beachten?
- Wie werden in C Strings realisiert?
- Was sind Structs und wie verwendet man diese?

45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie ihrem Sitznachbarn (45 Sekunden):

- Wie verwendet man Arrays in C und was ist zu beachten?
- Wie werden in C Strings realisiert?
- Was sind Structs und wie verwendet man diese?



45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie ihrem Sitznachbarn (45 Sekunden):

- Wie verwendet man Arrays in C und was ist zu beachten?
- Wie werden in C Strings realisiert?
- Was sind Structs und wie verwendet man diese?

45 Sekunden